

Assignment 2- Case Study

Stage 3 Lab Activities

SmartCare Domain Modelling

AI OFF -> AI ON -> COMPARE -> VERIFY | 1 hour

A - Requirements Review

Highlight nouns, verbs and business rules in SmartCare v0.2.

Nouns

Verbs

Business Rules

B - Candidate Classes

Record candidate concepts, supporting requirements, state and behaviour.

Candidate Concepts	Supporting Requirements	State	Behaviour
Patient	FR-01, FR-02, FR-03, FR-09, FR-12; US-01, US-05, US-06	Patient ID, name, patient details, stored information	Create record, search patient, update details, retrieve information
Practitioner	FR-04, FR-05, FR-06; US-02, US-04	Practitioner details, availability and schedule	View manage schedule along with patient details
Appointment	FR-04, FR-05, FR-06, FR-07, FR-08, FR-10, FR-12; US-02, US-03, US-04	Appointment details, date/time, patient, practitioner, status	Book/create appointment, cancel appointment, update status, release booking slot, retain appointment history
Schedule	FR-06, US-04	Scheduled appointments and available times	Display available appointment times and practitioners.
Report	FR-09	Report data	Generate operational reports

C - CRC Cards

Create CRC cards for Patient, Practitioner and Appointment.

Patient

Responsibilities	Collaborators
Store patient information	Appointment
Update patient information	Appointment

Practitioner

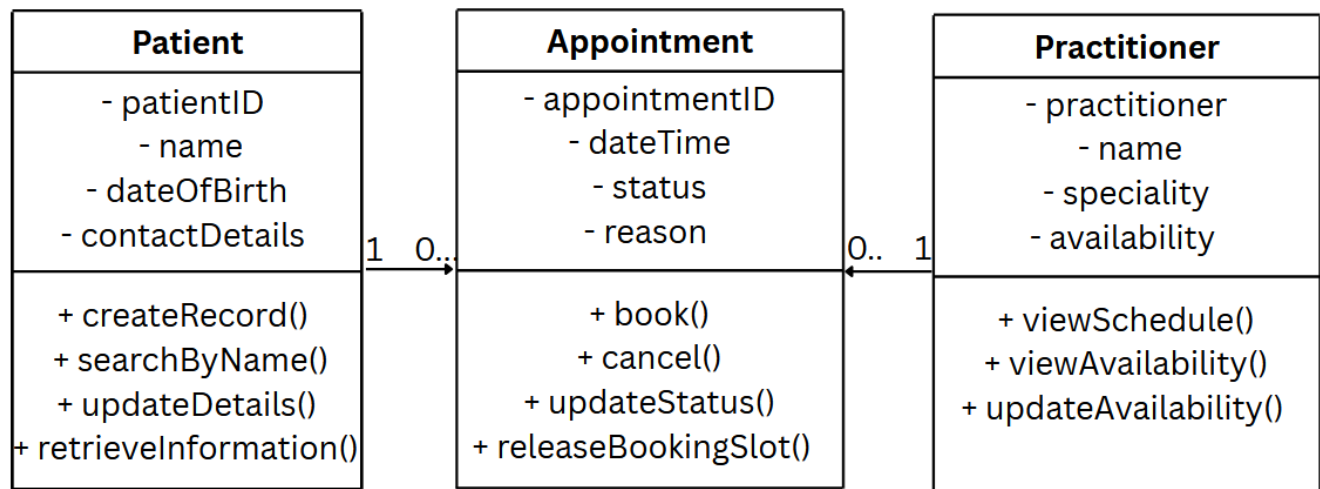
Responsibilities	Collaborators
Maintain practitioner availability	Appointment
Provide scheduled appointment information	Appointment

Appointment

Responsibilities	Collaborators
Create/book appointment	Patient/Practitioner
Prevent double booking	Patient/Practitioner

D - UML Model

Draw classes, attributes, operations, associations and multiplicities.



E - AI Design Review

Ask AI to suggest classes and relationships using only confirmed requirements; require supporting requirement IDs.

AI suggestion Supporting requirement ID(s) Decision Reason

Patient class	FR-01, FR-02, FR-03, FR-12	Accepted	Patient records must be created, searched, updated and retrieved.
----------------------	----------------------------	----------	---

Practitioner class	FR-04, FR-05, FR-06	Accepted	Appointments must be associated with practitioners and practitioner schedules/availability must be managed.
Appointment class	FR-04–FR-08, FR-10, FR-12	Accepted	The requirements directly describe creating, cancelling, updating and retrieving appointments.
Schedule	FR-06, US-04	Rejected	Schedule information is required, but the requirements do not establish it as an independent domain object.
Report	FR-09	Modified	Reporting is required, but FR-09 does not provide enough detail to justify a separate domain class.

F - Compare and Decide

Record at least one accepted, modified and rejected AI suggestion.

AI suggestion	Decision	Reason
Appointment class with booking, cancellation and status responsibilities	Accepted	Supported by FR-04, FR-07, FR-08 and FR-10. Appointment is a clear core domain concept required by the system.
Schedule as a separate class	Modified	FR-06 and US-04 confirm that schedules and availability are required, but there is insufficient evidence that Schedule should be a separate class. Schedule information will instead be represented through Practitioner and Appointment.
NotificationManager class	Rejected	No confirmed requirement supports notifications, reminders or a notification-management function. Adding this class would introduce functionality outside the defined scope.

G - Python Skeletons

Create simple Patient, Practitioner and Appointment class skeletons.

Pls see Source Code

H - Consistency Check

Check model-code consistency; do not implement full behaviour yet.

Model element	Python Implementation	Consistent
Patient Class	Class Patient	Yes
Patient ID	patient_id	Yes
Patient Name	name	Yes
Date of Birth	update_details	Yes
Contact Details	update_details, retrieve_information	Yes
Appointment Class	Class Appointment	Yes
Appointment ID	appointment_id	Yes
Date/Time	date_time	Yes
Status	“booked”	Yes
Reason	self.reason_for_visit	Yes
Practitioner Class	Practitioner Class	Yes
Practitioner	practitioner_id, name	Yes
Name	name	Yes
Specialty	self.specialisation	Yes
Availability	view_availability, view_schedule	Yes

Reflection

What modelling decision was hardest? Where did AI over-design? What evidence supported your final choices?

The hardest modelling decision was deciding which candidate concepts should become classes. In particular, I considered whether Schedule and Report should be separate classes. Both concepts appear in the requirements, but the requirements do not provide enough detail to justify them as independent domain classes. I therefore decided to represent schedule and availability information through Practitioner and Appointment rather than creating additional classes.

AI over-designed the system by suggesting additional management and support classes, such as Manager, Controller and NotificationManager. These could be useful in a larger software system, but they were not stated to be supported by the confirmed SmartCare requirements. Adding them would increase the complexity of the model without providing evidence that they were needed.

My final choices were supported by the functional requirements, user stories and business rules. Patient was supported by requirements for creating, searching, updating and retrieving patient records (FR-01, FR-02, FR-03 and FR-12). Practitioner was supported by requirements concerning practitioner availability and appointments (FR-04, FR-05 and FR-06). Appointment was strongly supported by the booking, cancellation, status and double-booking requirements (FR-04 to FR-08 and FR-10). This evidence-based approach kept the final model simple and within the scope of the SmartCare case study.